

WELCOME TO

Software Engineering & Testing

Introduction Class

BY PREMITHA KAMATH



What is a Software?



“Computer programs and associated documentation.”



What is software?

Software is a collection of programs, data, and instructions that tell a computer or digital device what to do and how to do it.



Types

- **System Software:** Runs and manages the computer itself. Examples: Windows, macOS, Linux, Android
- **Application Software:** Helps users perform specific tasks. Examples: Word processors, web browsers, games, music players
- **Programming Software:** Used to create other software. Examples: Compilers, code editors, IDEs



Imagine
Life
without
Software!

What are the attributes of a good software?

Product characteristic	Description
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable, and compatible with other systems that they use.
Dependability and security	Software dependability includes a range of characteristics including reliability, security, and safety. Dependable software should not cause physical or economic damage in the event of system failure. Software has to be secure so that malicious users cannot access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, resource utilization, etc.
Maintainability	Software should be written in such a way that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

Zerodha's Nithin Kamath launches Kite Backup for trading: What it is, how to use the service – 10 things to know

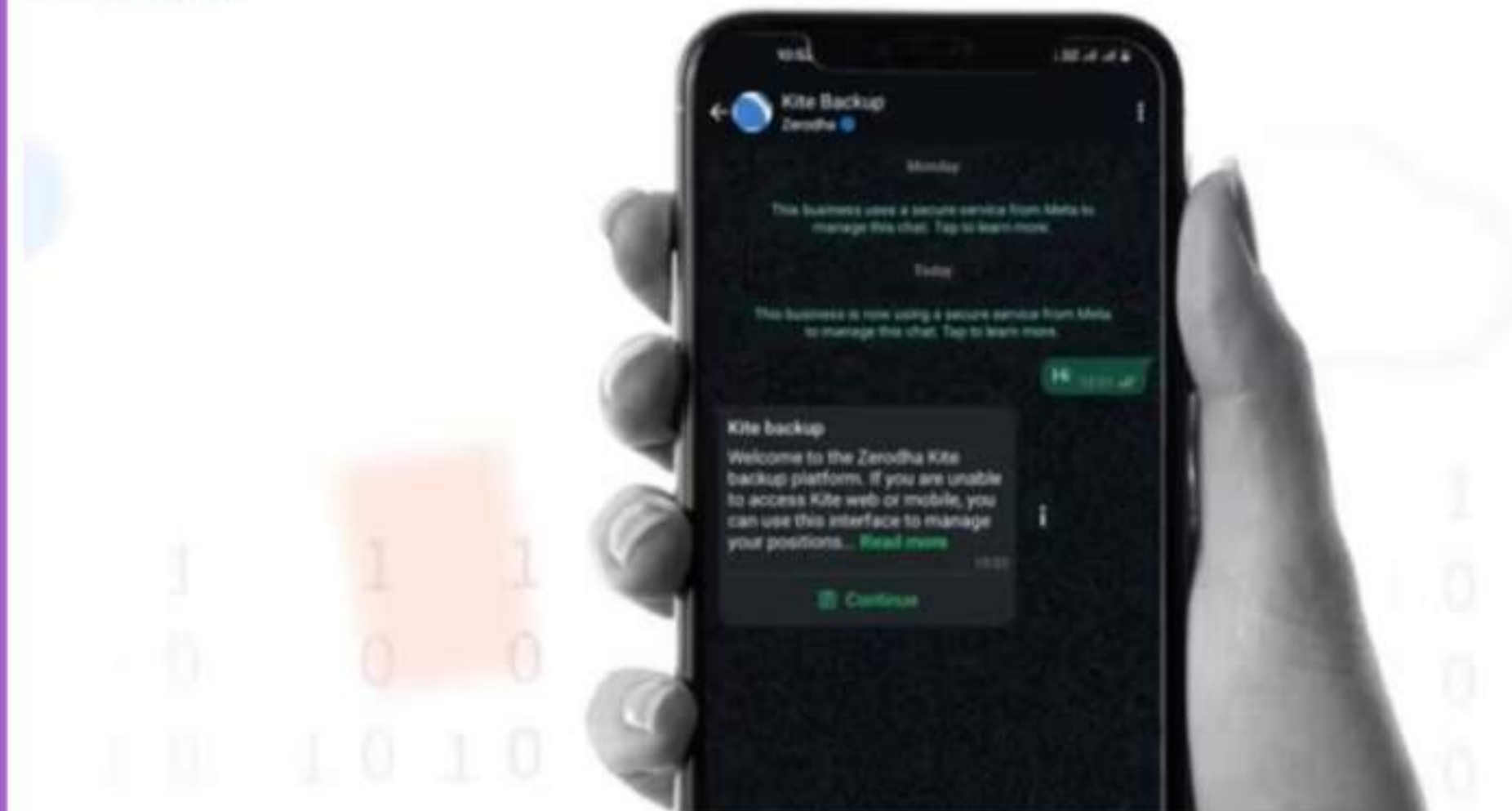
Zerodha founder and CEO Nithin Kamath on August 14 introduced Kite Backup, an emergency trading model that aims to enhance client safeguards. It allows users to cancel orders via WhatsApp during outages of the Kite platform (website and mobile app).

Jocelyn Fernandes

Updated • 14 Aug 2025, 06:51 PM IST



 ZERODHA



Software Processes

A software process is a set of related activities that leads to the production of a software system.

There is no universally applicable software process.

However, although there are many different software processes, they all must include, in some form, the four fundamental software engineering activities



Software specification

The functionality of the software and constraints on its operation must be defined



Software development

The software to meet the specification must be produced.



Software validation

The software must be validated to ensure that it does what the customer wants.

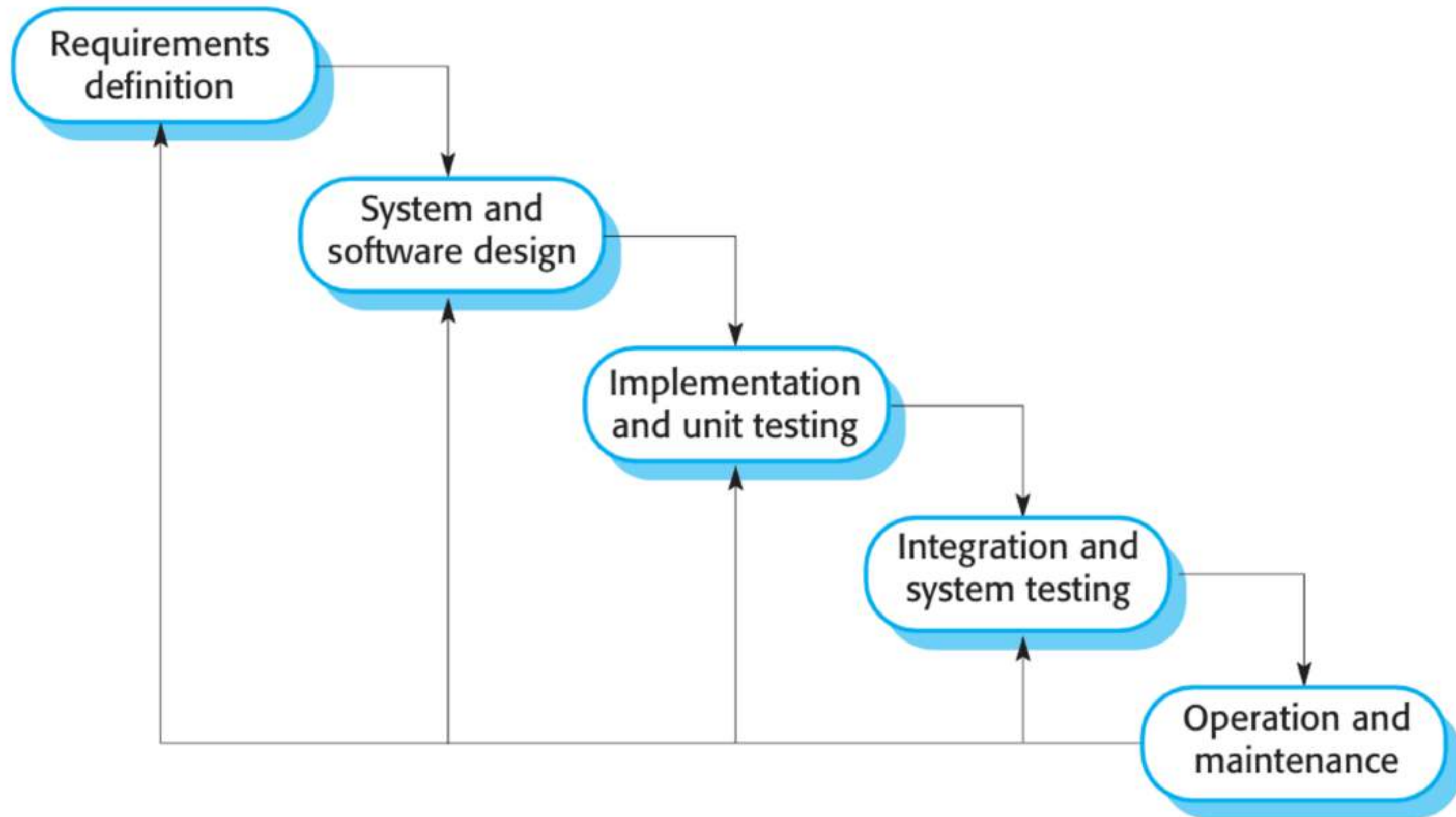


Software evolution

The software must evolve to meet changing customer needs.

Software Development Lifecycle (SDLC) Models





Waterfall Model

The waterfall model is a **linear, sequential software development process** where development flows through a fixed set of stages. **Each stage must be completed and approved before the next begins.**



Requirements Analysis and Definition

Requirements are documented in detail and become the system specification.



System and Software Design

Requirements are allocated to hardware or software. Overall system architecture is defined.



Implementation and Unit Testing

Software design is converted into code. Individual program units are tested to ensure they meet specifications.



Integration and System Testing

All units are integrated into a complete system. System-level testing ensures requirements are satisfied



Operation and Maintenance

The system is deployed and used in real environments. Errors are fixed, performance improved, and new features added. This is usually the longest phase of the life cycle.¹¹

Limitations of Waterfall model



- 01 Inflexible to Change
- 02 Requirements Must Be Clear Upfront
- 03 Late Testing and Feedback
- 04 No Early Working Software
- 05 It is not suitable for large, complex, or high-risk projects where requirements and technology may change frequently.
- 06 Limited Customer Involvement
- 07 Documentation-Heavy



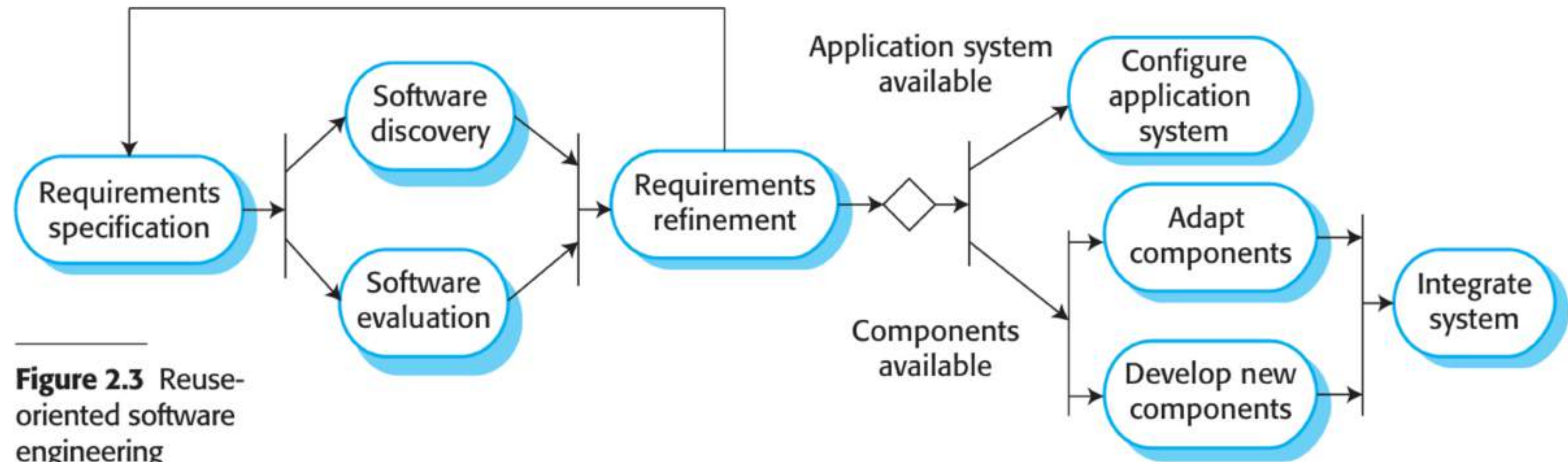


Figure 2.3 Reuse-oriented software engineering

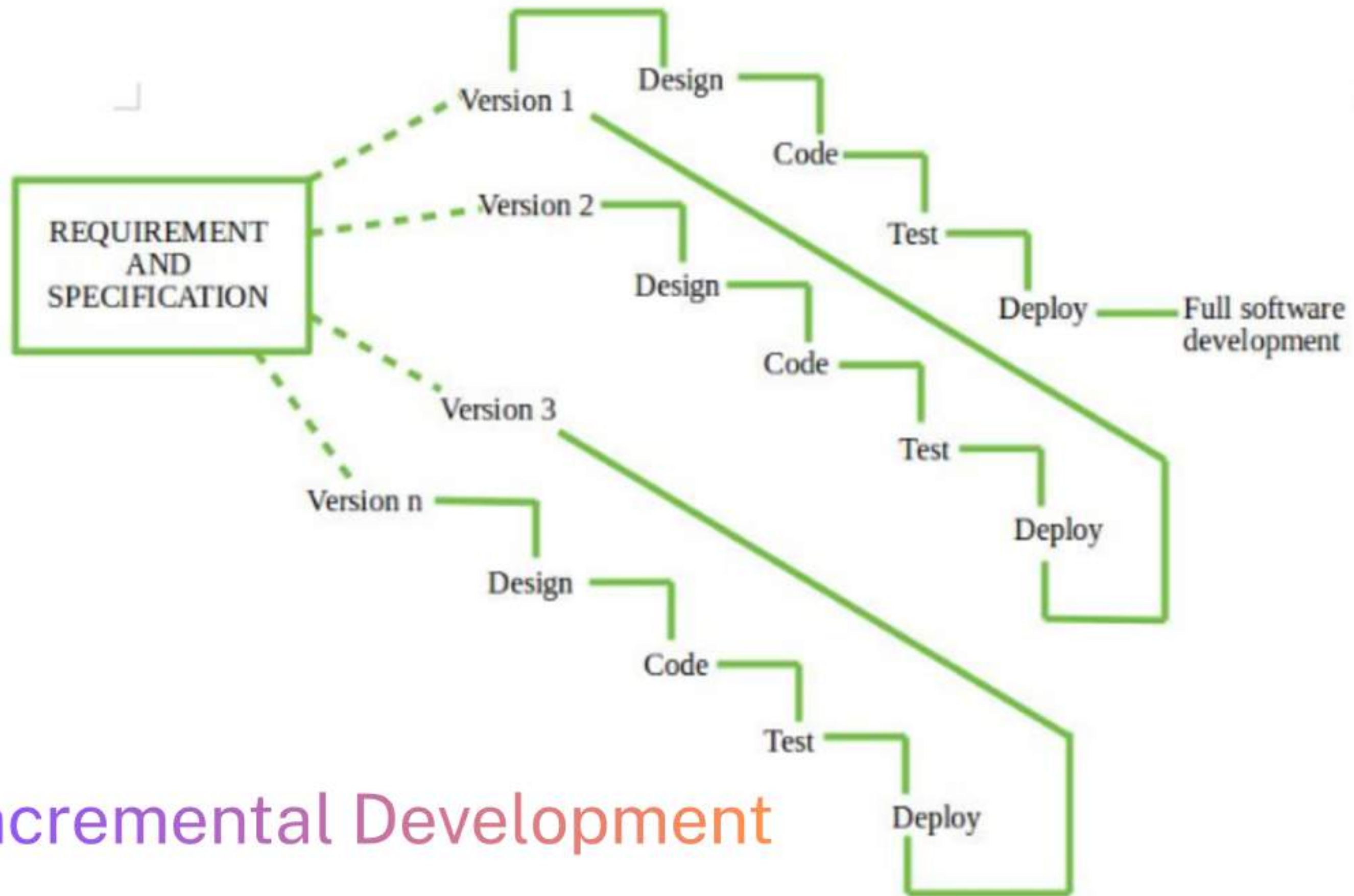
Integration & Configuration

In most software projects, some level of software reuse occurs. Traditionally, this reuse has been informal, where developers:

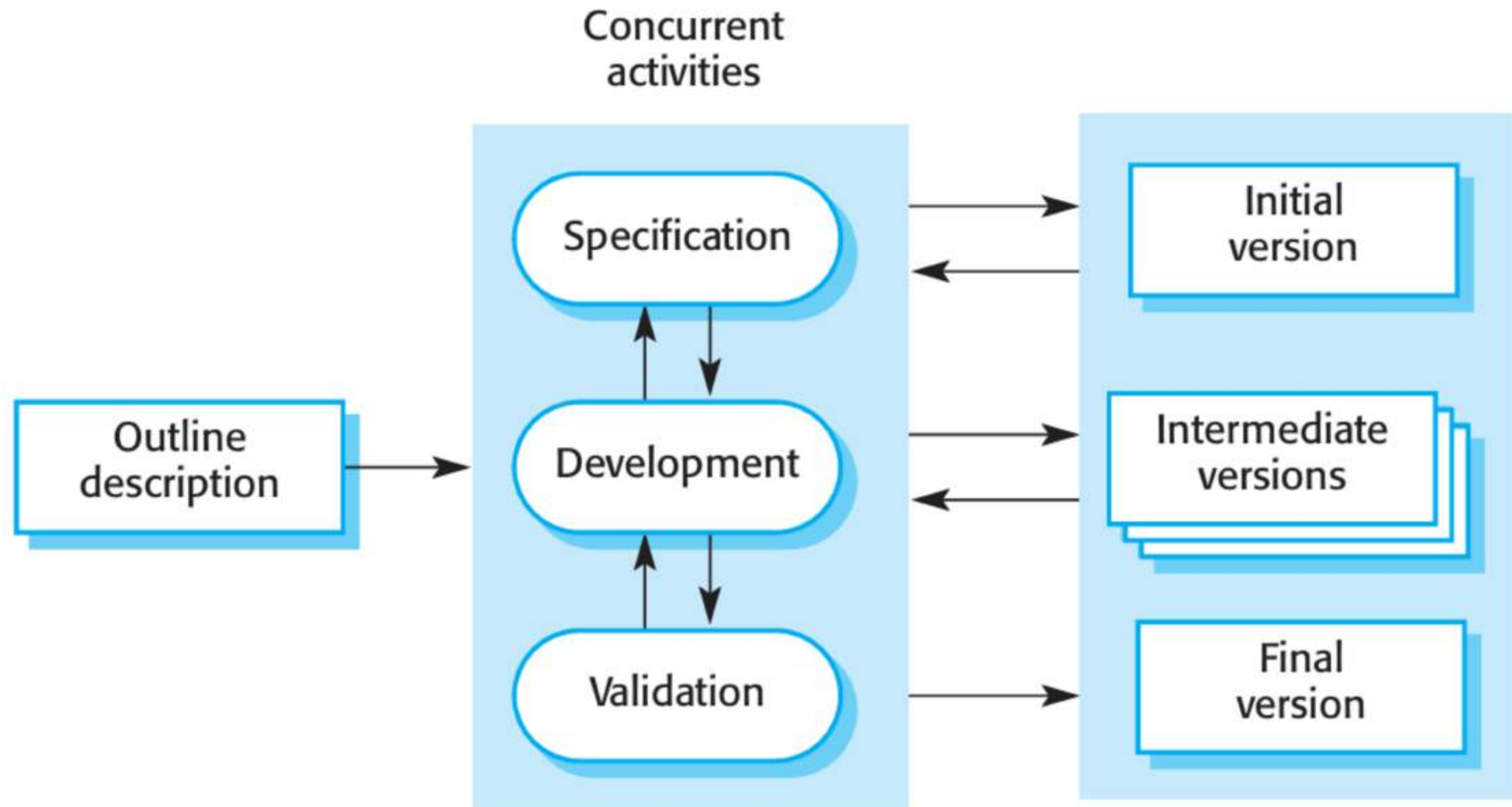
- Search for existing code
- Modify it to meet current needs
- Integrate it with newly written code

Reuse-Oriented Development Process

- **Requirements Specification**
 - High-level requirements are defined
 - Focus on essential and desirable features
 - Detailed specification is not required at this stage
- **Software Discovery and Evaluation**
 - Search for existing systems or components
 - Evaluate candidates against essential requirements
 - Assess suitability for integration
- **Requirements Refinement**
 - Requirements are adjusted based on available components
 - System specification is redefined
 - If compromises are unacceptable, alternative components are sought
- **Application System Configuration**
 - If a suitable off-the-shelf system exists, it is configured
 - Configuration creates the new system without major development
- **Component Adaptation and Integration**
 - If no complete system exists:
 - Reusable components are adapted
 - New components are developed where necessary
 - All components are integrated into the final system



Incremental Development



Incremental Development

Incremental development is a software process where an **initial working version of a system is built and then evolved through multiple versions (increments).**

Each increment adds new functionality based on user feedback and changing requirements.



Key Characteristics

Software is developed in small, manageable increments.
User feedback is collected early and frequently.
Requirements can evolve during development
Can be:

- **Plan-driven:** all increments planned in advance
- **Agile:** only early increments planned; later ones depend on priorities
- **Hybrid:** a mix of both (most common)



How Incremental Development Works

- Each increment delivers a subset of system functionality
- Early increments implement core or high-priority features
- Users evaluate early versions and provide feedback
- Changes usually affect only the current or future increments, making them cheaper and easier to implement



Agile Software Development

Presented By:
Premitha Kamath

What is Agile?



What is Agile?

 **agile** ['adʒaɪl]

Adjective

1. able to move quickly and easily:

"Ruth was remarkably agile and light on her feet" · "the changes were needed for the company to become more agile and move faster"

Similar:

nimble

lithe

spry

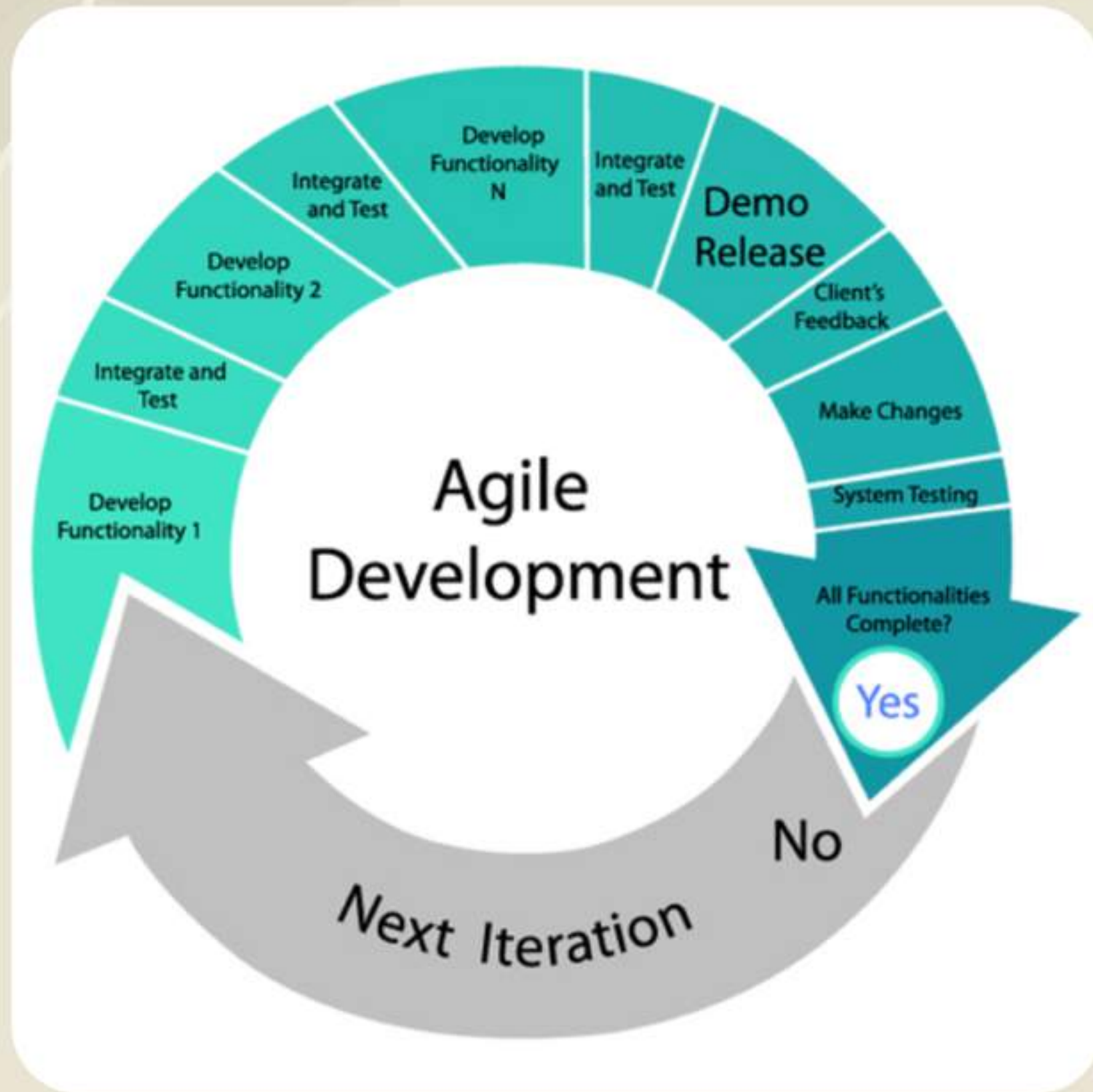
supple

limber

sprightly

- **able to think and understand quickly:**

"his vague manner concealed an agile mind"



What is Agile Software Development?

Agile Methods in Software Engineering refer to a group of modern development approaches that emphasize **flexibility, customer collaboration, iterative progress, and rapid delivery of working software**. Agile was formally introduced through the Agile Manifesto (2001).



🔍 **Fits Fast-Changing Technology & Markets** – Need to react to changes more quickly than 2 yearlong waterfall projects

🔍 **Reduces Risk of Failure** – Need to react to changes more quickly than 2 yearlong waterfall projects.

✦ 🔍 **Better Quality Through Continuous Testing** – Testing happens continuously, Bugs are found early, Quality improves over time

Why Agile ?



Good agile collaboration feels like the swing rowing crew members achieve.

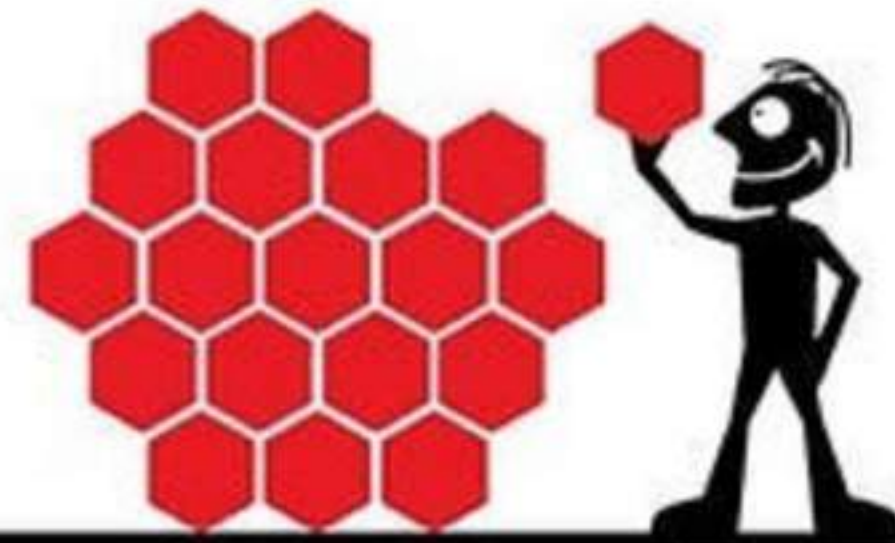
Goal- Deliver working software quickly

THE WATERFALL PROCESS



*'This project has got so big,
I'm not sure I'll be able to deliver it!'*

THE AGILE PROCESS



*'It's so much better delivering this
project in bite-sized sections'*

Agile's main goal is to give users usable software as early as possible, instead of waiting until everything is finished.

Agile

Compromise – Less functionality, not lower quality

Agile does not reduce quality.

- Each delivery has fewer features
- But the features delivered are fully tested and high quality

Focus on the code rather than the design

Agile does not spend months on detailed upfront design.

- Design evolves as the code evolves
- The team improves the design continuously

Less documentation

Agile avoids heavy, detailed documents that take a lot of time.

- Documentation exists
- But working software is more important
- Requirements change
- Old documents become useless quickly



Agile

Interleave (Do things together)

In Agile, activities do not happen in strict phases.

Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.

Instead of:

1. Finish specification
2. Then design
3. Then coding

Agile does:

- Small specification
- Small design
- Small implementation
- Repeated again and again (in sprints)

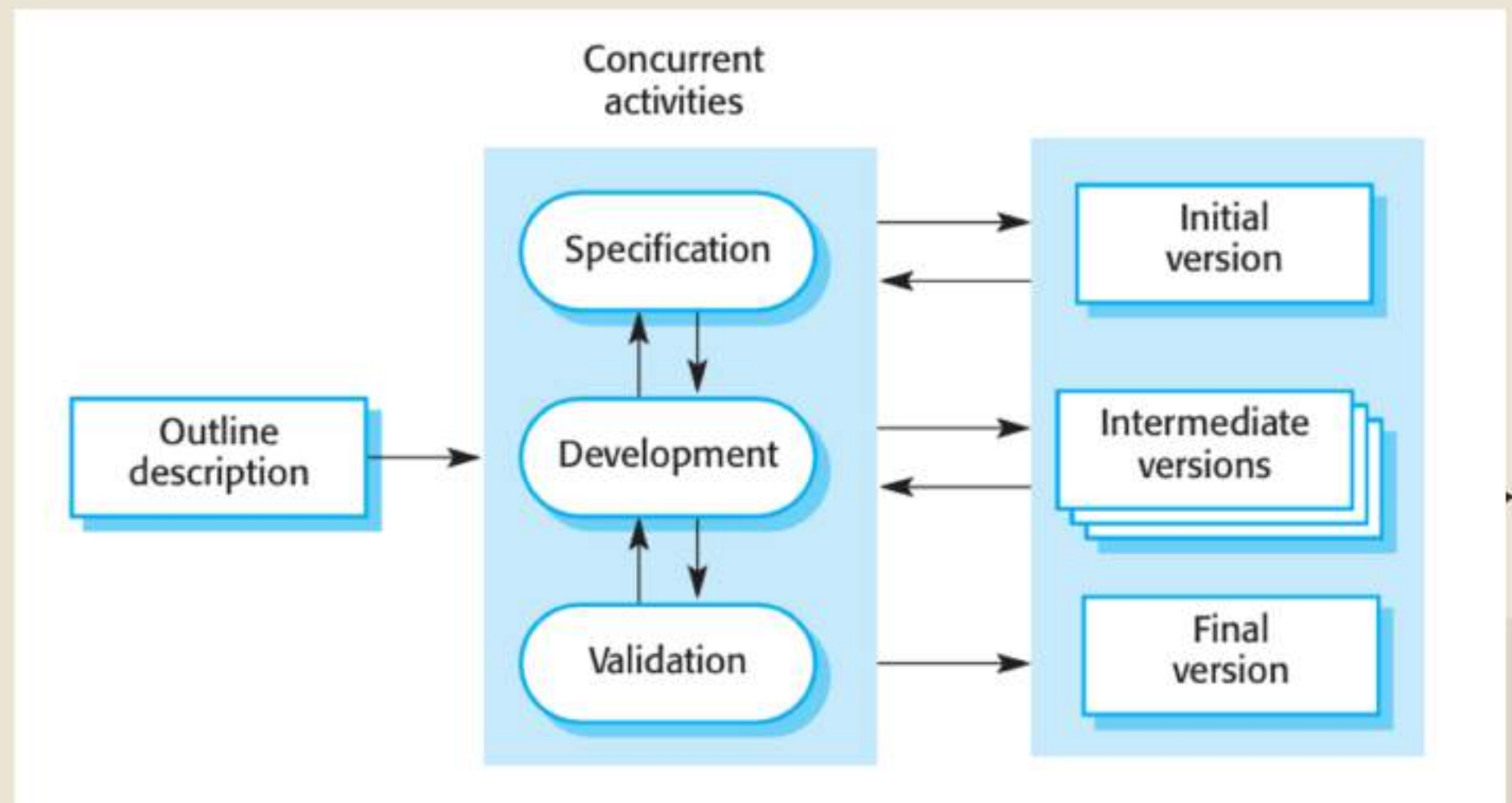
Deliver small versions and get user (stakeholder) input

Agile releases:

- Small versions frequently

After each release:

- Users give feedback
- Stakeholders suggest changes
- Product improves continuously



Agile Principles

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

- ✓ **Individuals and interactions over processes and tools**
- ✓ **Customer collaboration over contract negotiation**

- ✓ **Responding to change over following a plan**
- ✓ **Working software over comprehensive documentation**



Agile Terminologies

01 ITERATIVE AND INCREMENTAL DEVELOPMENT

SMALL CYCLES

Building software in small, manageable iterations.

02 CONTINUOUS INTEGRATION AND CONTINUOUS DELIVERY (CI/CD):

Frequent code integration with automated testing and deployment.

03 TEST-DRIVEN DEVELOPMENT (TDD)

Writing tests before code to ensure reliability.



Agile Terminologies

04 PAIR PROGRAMMING

Two developers work together to improve code quality and knowledge sharing.

05 REFACTORING

Continuously improving code structure without changing functionality.

06 DAILY STAND-UPS AND RETROSPECTIVES

Regular meetings to track progress and improve team performance.



Agile Terminologies

07 USER STORIES

Simple descriptions of features from the user's perspective

User Story

As a <role>
I want <goal>
so that <benefit>

Acceptance criteria:
(Conditions of Satisfaction)

...
...

As an Account Manager
I want a sales report of my account
to be sent to my inbox daily
So that I can monitor the sales
progress of my customer portfolio

Acceptance criteria:

1. The report is sent daily to my inbox
2. The report contains the following sales details: ...
3. The report is in csv format.

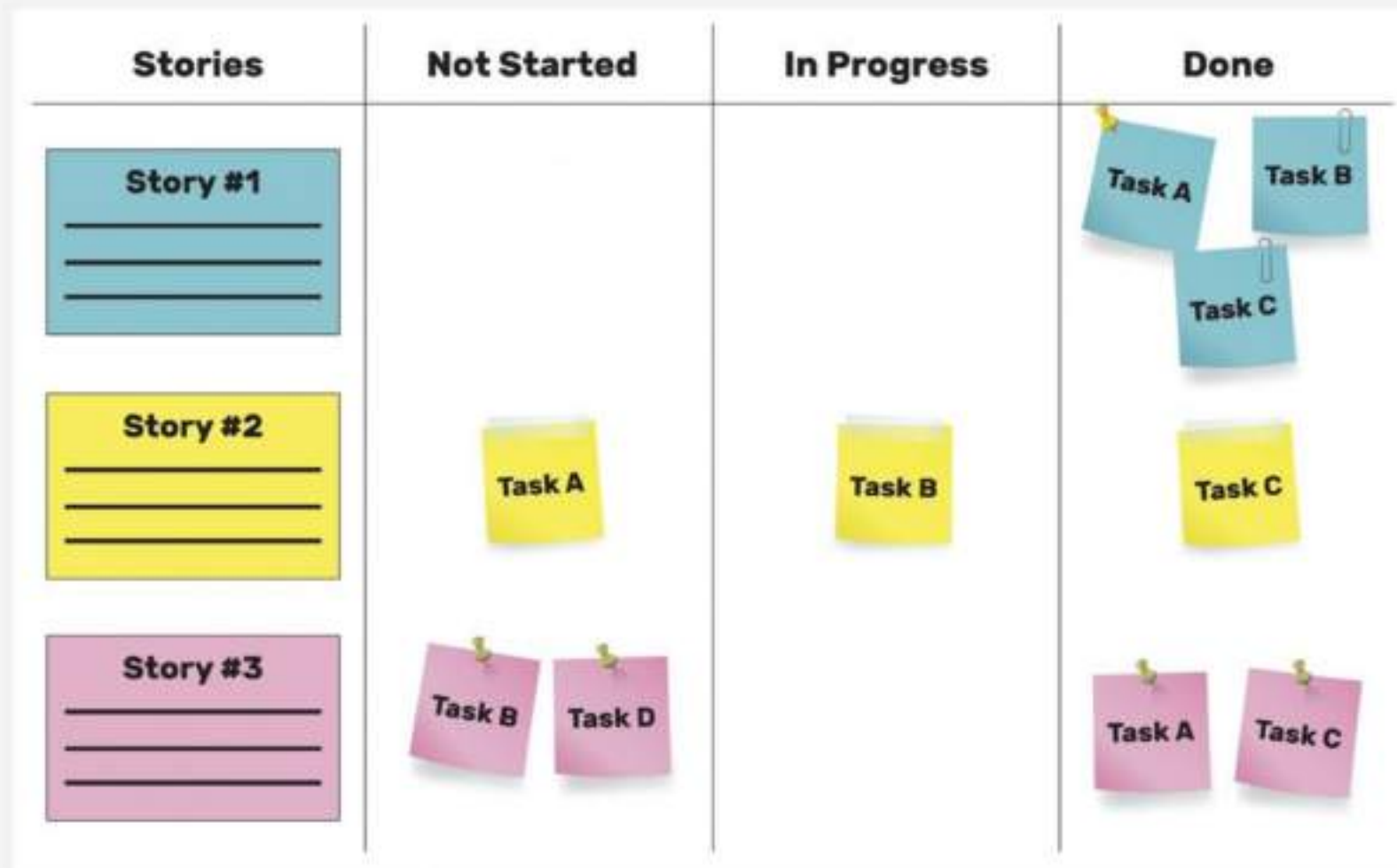


User stories for Requirements

User requirements are expressed as user stories or scenarios.

These are written on cards, and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.

The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.



Prescribing medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose; If she wants to change the dose, she enters the new dose then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Prescribing medication story

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, look up the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

Agile Development Techniques

Premitha Kamath



EXTREME PROGRAMMING XP

XP takes good software practices and pushes them to extreme levels

WHY THE NAME EXTREME PROGRAMMING?

XP takes good software practices and pushes them to extreme levels:

- **Iterative development → Very short iterations**
- **Testing → Test before coding**
- **Integration → Several times per day**
- **Feedback → Continuous**

EXTREME PROGRAMMING XP

XP takes good software practices and pushes them to extreme levels

HISTORY

- Agile ideas started to emerge in the 1990s .
 - The most influential approach was Extreme Programming (XP).
 - XP was introduced by Kent Beck (1998).
-

EXTREME PROGRAMMING XP

XP takes good software practices and pushes them to extreme levels

HOW EXTREME PROGRAMMING (XP) WORKS

Requirements as User Stories

- Requirements are written as user stories (scenarios).
- Example: “As a user, I want to reset my password so I can access my account.”
- Each story is broken into small tasks.

• Pair Programming

- Developers work in pairs:
 - One writes code
 - One reviews continuously
- Improves:
 - Code quality
 - Knowledge sharing
 - Fewer defects

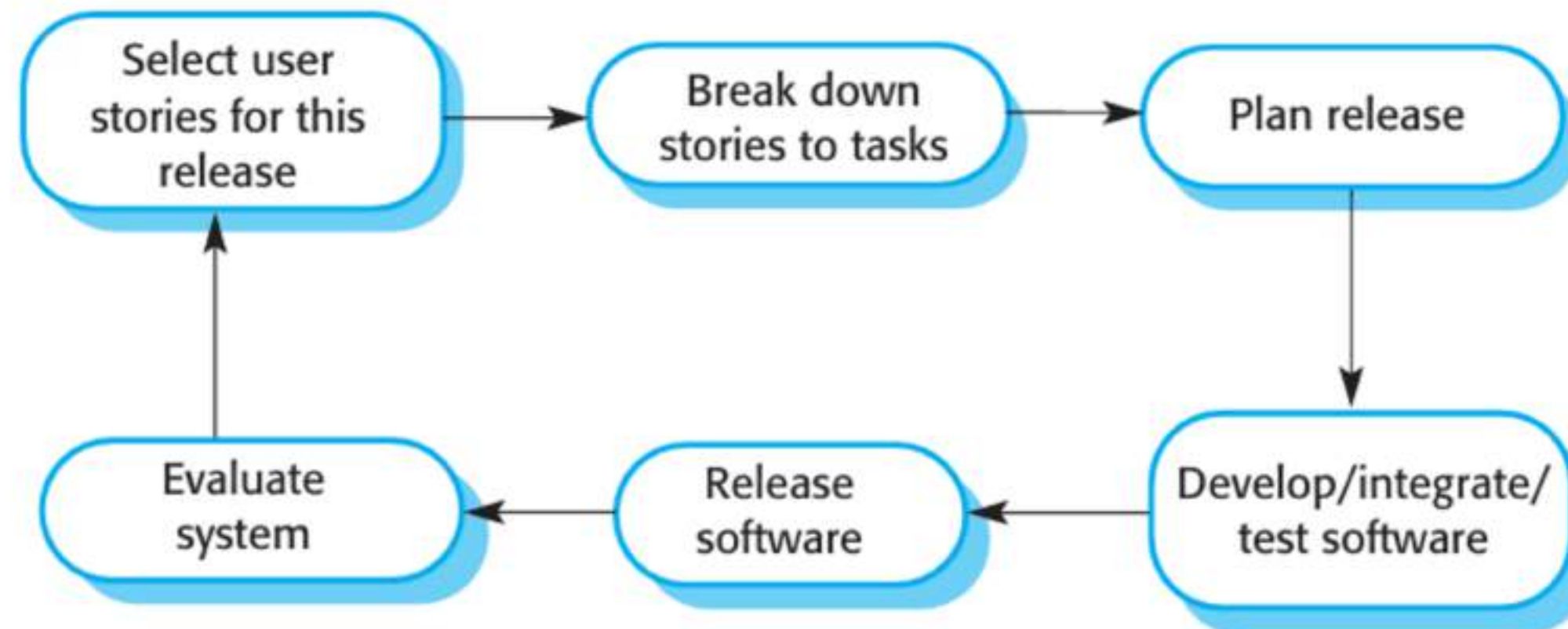
• Test-First Development & Frequent releases

- Tests are written before the code.
- Code is accepted only if all tests pass.
- Time between releases is very short (often 1–2 weeks).
- Customers see working software early and often.

EXTREME PROGRAMMING XP

XP takes good software practices and pushes them to extreme levels

THE XP RELEASE CYCLE



EXTREME PROGRAMMING XP

EXTREME PROGRAMMING PRACTICES

Principle or practice	Description
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Incremental planning	Requirements are recorded on "story cards," and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development "tasks." See Figures 3.5 and 3.6.
On-site customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.

EXTREME PROGRAMMING XP

EXTREME PROGRAMMING PRACTICES

Refactoring	All developers are expected to refactor the code continuously as soon as potential code improvements are found. This keeps the code simple and maintainable.
Simple design	Enough design is carried out to meet the current requirements and no more.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Sustainable pace	Large amounts of overtime are not considered acceptable, as the net effect is often to reduce code quality and medium-term productivity.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.

EXTREME PROGRAMMING (XP)

REFACTORING IN XP: Traditional software engineering promotes “designing for change”, meaning systems are built with flexibility and generality to accommodate future changes. **Extreme Programming (XP) challenges this idea.**

Traditional Approach:

- Anticipate future requirements.
- Add generality and flexibility early.
- Risk: effort may be wasted if predicted changes never occur.

Extreme Programming (XP):

- Avoid designing for hypothetical future changes.
- Focus on current requirements only.
- Belief: most anticipated changes never happen or are different from what was expected.

Role of Refactoring in XP

Since change is inevitable, XP relies on continuous refactoring instead of upfront design for change.

Refactoring means:

- Improving internal code structure without changing behavior
- Done continuously during development
- **Examples:**
 - Removing duplicate code
 - Renaming variables and methods for clarity
 - Reorganizing class hierarchies
 - Replacing repeated logic with reusable methods



`<dev /> speak`

teaching non-devs to speak like a dev

Refactor

Meaning

Kitty sees bad code. Kitty decides to fix. She rewrites the function. Fix variable names. Make code read better. No change to features, just better code. That's refactoring.

Usage

"Let's spend some extra time today to refactor some old code." - said no one ever



EXTREME PROGRAMMING XP

TEST-FIRST DEVELOPMENT

Testing in XP

- In Extreme Programming, testing is not a final phase.
- The system is tested after every single change.
- A change is not accepted unless all tests pass.

Why XP Emphasizes Testing

- Frequent changes increase risk of bugs
- **Continuous testing:**
 - Detects errors early
 - Prevents broken code from spreading
 - Maintains high software quality

EXTREME PROGRAMMING XP

TEST-FIRST DEVELOPMENT

Test-First Development (TDD)

- Tests are written before the actual code.
- Code is written only to make the test pass.

Example

- Write a test:
 - “Password must be at least 8 characters”
- Then write code until the test succeeds

Incremental Test Development from Scenarios

- Tests are derived from user stories (scenarios).
- Each scenario leads to one or more tests.

Example User Story

“As a user, I want to transfer money so that I can pay my bills.”

Derived Tests

- Transfer succeeds with valid balance
- Transfer fails if balance is insufficient

EXTREME PROGRAMMING XP

TEST-FIRST DEVELOPMENT

User Involvement in Testing

- Customers help define **acceptance tests**.
- Ensures the system behaves as users expect.

Example

- Customer defines:
 - What “successful payment” means
 - What error messages should appear

Automated Test Harnesses

- XP uses automated testing tools.
- All component tests run:
 - Every time code is integrated
 - Every time a new release is built
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
- Usually relies on a testing framework such as Junit.

What Is a Test Harness?

- A framework that:
 - Runs tests automatically
 - Reports failures instantly

Prescribing medication – Dose Checking

Test 4: Dose checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

PAIR PROGRAMMING



**IN XP, PROGRAMMERS WORK IN PAIRS,
SITTING TOGETHER TO DEVELOP CODE.**

COMMON OWNERSHIP

KNOWLEDGE SPREAD

INFORMAL REVIEW

REFACTORING

**SIMILAR OUTPUT TO TWO PEOPLE
CODING**

PAIR PROGRAMMING



- Pair programming involves programmers working in pairs, developing code together.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from improving the system code.
- In pair programming, programmers sit together at the same computer to develop the software.
- Pairs are created dynamically so that all team members work with each other during the development process.
- The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.



SCRUM

By Premitha Kamath

AGILE METHADODOLOGLIES



HISTORY



- In 1986, Takeuchi and Nonaka published “**The New Product Development Game**” in Harvard Business Review.
- They compared high-performing product teams to a **rugby scrum**:
 - The team moves together
 - No rigid hand-offs
 - Continuous collaboration



SCRUM DEFINITION



Scrum is an Agile method that delivers software incrementally through sprints, starting with high-level planning and ending with project review and documentation.

SCRUM HAS THREE MAIN PHASES

Project Closure Phase

03

- The project is finalized and delivered.
- Necessary documentation (user manuals, help guides, etc.) is completed.
- The team reviews the project to identify lessons learned for future improvement.

Sprint Cycles (Development Phase)

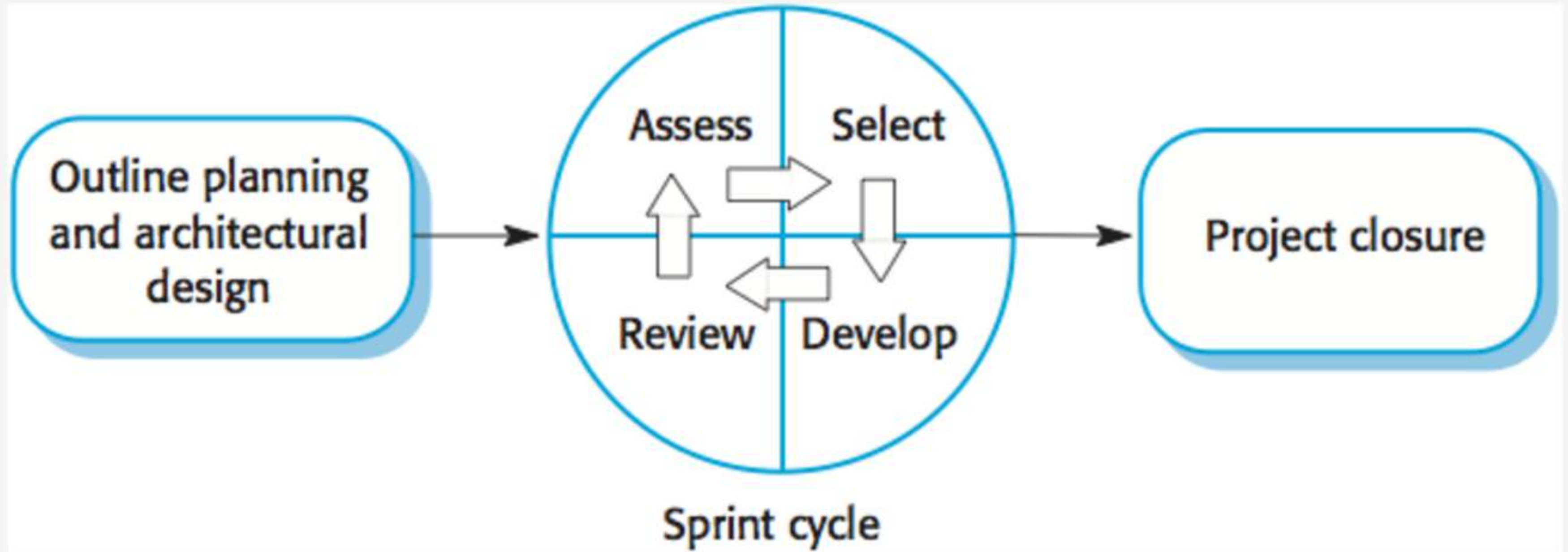
02

- The project is developed through short iterations called sprints.
- Each sprint produces a working part (increment) of the system.
- Feedback is reviewed after every sprint, allowing improvements and changes.

Planning

01

- The team defines the overall goals of the project.
- A basic plan and software architecture are created.
- This phase sets the direction but does not plan everything in detail.



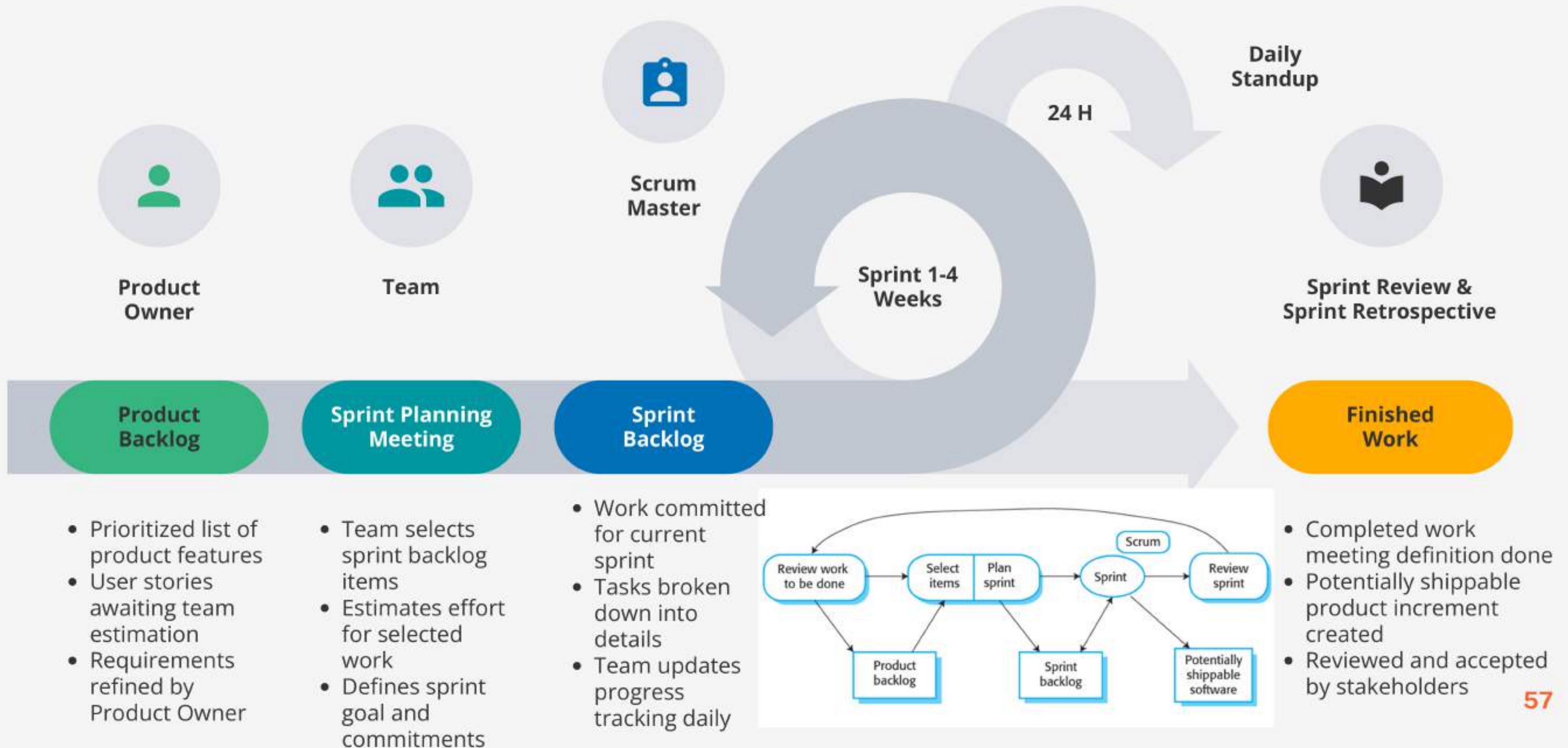
SCRUM TERMINOLOGIES

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than seven people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be “potentially shippable,” which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of “to do” items that the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories, or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development, and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

SCRUM TERMINOLOGIES

Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2 to 4 weeks long.
Velocity	An estimate of how much product backlog effort a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

SCRUM PROCESS DIAGRAM



SCRUM BENEFITS

Scrum improves flexibility, communication, customer satisfaction, and team trust by delivering work in small, visible increments.



The product is divided into small, manageable pieces, making it easier to understand and develop.



Changing or unclear requirements do not stop the project, because work continues in short iterations.



The entire team has full visibility of the project, which improves communication and collaboration



Customers receive working parts of the product on time and can give feedback early and often.



Continuous interaction builds trust between customers and developers, creating a positive team culture where everyone is confident about the project's success.

Kanban Board

By Premitha Kamath



A way to visualize our workflow to maximize efficiency.

Project Kanban Board

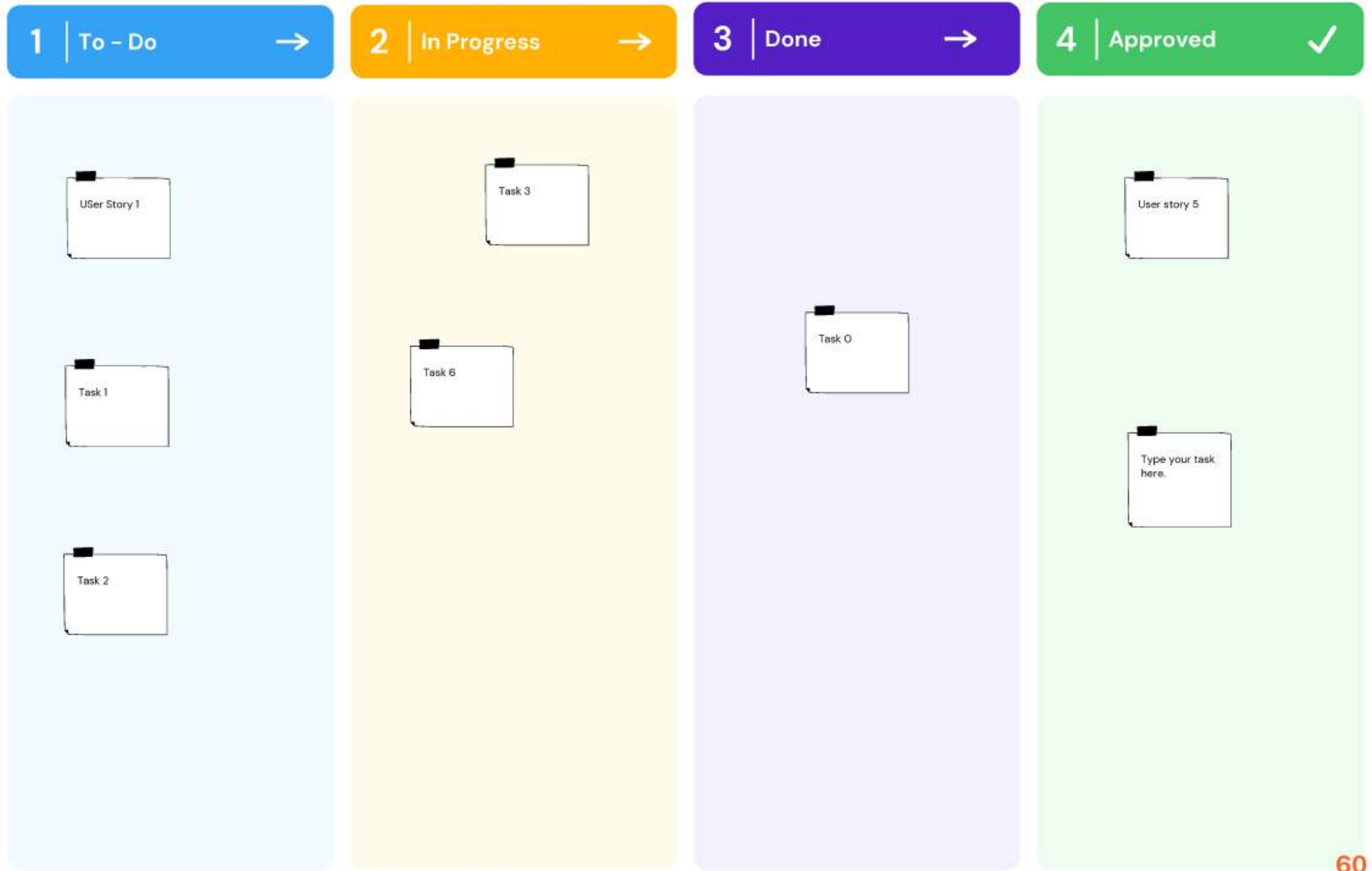
WEEK 01 | June 01 - 05

1 Copy a sticky note, then type in the task that needs to be done.



2 Drag and drop to the corresponding column.

3 Transfer task to the next column as its status updates.



2 | In Progress



Visual workflow
management
using a Kanban
board

Limits work in
progress (WIP)
to improve
flow and
reduce
bottlenecks

Encourages
flexibility and
efficiency
without fixed
iterations

Work is shown
as cards
moving
through stages
(To Do → In
Progress →
Done)

Focuses on
continuous
delivery and
incremental
improvement

Problems with Agile Methods

01

DIFFICULT TO KEEP CUSTOMERS/USERS INTERESTED AND INVOLVED

Difficult to keep customers/users interested and involved

A client agrees to attend sprint review meetings every two weeks. After a few months, they stop joining calls or give delayed feedback.

02

TEAM MEMBERS MAY BE UNSUITED TO INTENSE INVOLVEMENT

Agile demands constant collaboration, communication, and adaptability. Not everyone is comfortable with this.

A developer who prefers clear instructions and long-term planning struggles in daily stand-ups and frequent requirement changes.



Problems with Agile Methods

03

PRIORITIZING CHANGES IS DIFFICULT WITH MULTIPLE STAKEHOLDERS

Different stakeholders often have conflicting priorities, making decision-making harder.

- Marketing wants new promotional features.
- Finance wants cost-reporting tools.
- Operations wants performance improvements.

The product owner struggles to decide what goes into the next sprint, slowing progress and causing conflict.

04

MAINTAINING SIMPLICITY REQUIRES EXTRA WORK

Agile promotes simple solutions, but keeping systems simple while adding new features takes effort.

Instead of adding quick hacks to meet deadlines, developers must refactor code regularly to keep it clean. This extra refactoring takes time and discipline, which teams may skip under pressure.



Problems with Agile Methods

05

CONTRACTS MAY BE A PROBLEM

Traditional contracts expect fixed scope, cost, and deadlines, while Agile welcomes changing requirements.

- A contract says “Build Feature X exactly as described.”
- Midway through the project, the customer wants changes based on new insights.

This can lead to disputes, renegotiation, or legal issues.

06

DIFFICULT TO SCALE AGILE TO LARGE SYSTEMS

Agile works best with small, tightly integrated teams. Large systems need coordination across many teams.

A bank system involves 15 Agile teams working on different modules. Synchronizing sprint schedules, dependencies, and integration becomes complex and slows development.



Problems with Agile Methods

07

LESS EMPHASIS ON DOCUMENTATION

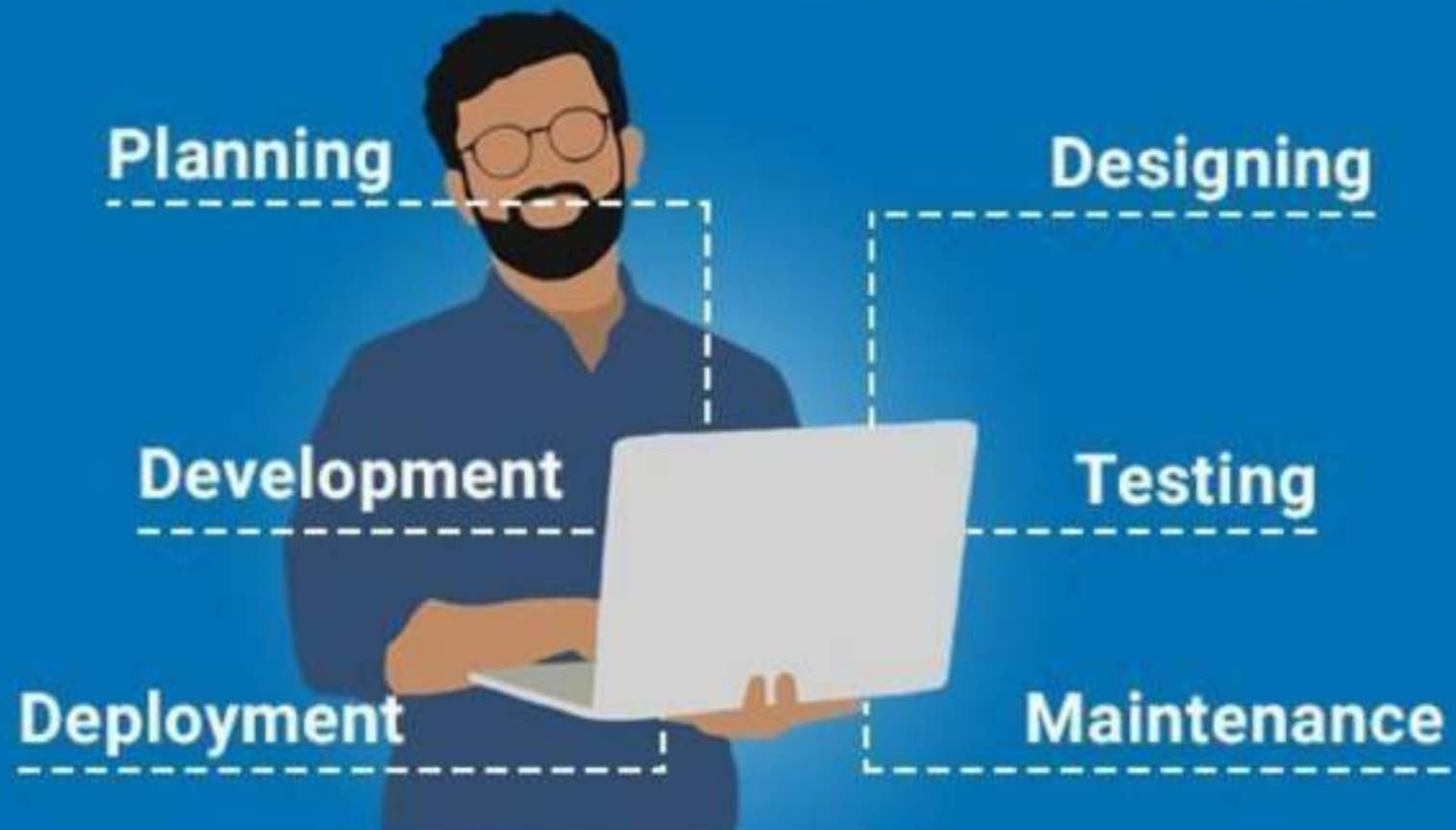
Agile focuses more on working software than documentation, which can hurt long-term maintenance.

- The original development team leaves after delivery.
- A new maintenance team joins but finds minimal documentation.

They struggle to understand the system, increasing bugs and maintenance costs.



Software Engineering



Thank you!

More in Next Class..